



University of Engineering and Technology
Lahore, Pakistan

Department of Computer Science
Session 2023 – 2027

MediQuick - Pharmacy Management System

Test Case Documentation

Submitted By
Amir Hashmi 2023-CS-11
Harris Amin 2023-CS-160

Supervised By
Dr. Rabeeya Saleem

Course: Mobile App Development

May 28, 2026

Contents

1	Overview	2
2	Test Cases	3
2.1	User Sign-Up Flow	3
2.2	User Sign-In Flow	4
2.3	Admin Hardcoded Login	4
2.4	Add New Medicine (Pharmacist)	5
2.5	Edit Existing Medicine	6
2.6	Billing — Add to Cart and Checkout	7
2.7	Sales History — View and Update Order Status	8
2.8	Low Stock Alerts	9
2.9	Add Pharmacist (Admin Flow)	10
2.10	Pharmacist Performance Stats (Admin View)	11
3	Test Summary Matrix	12

Overview

This document describes **10 functional test cases** for the MediQuick Android application. The app supports three user roles — **Admin**, **Pharmacist**, and **Customer (User)** — and integrates Firebase Authentication, Firestore, and a local Room database for offline-first data management.

Flows Covered

- Authentication — Sign Up, Sign In, Admin Login
- Medicine Inventory — Add, Edit, Delete
- Billing & Cart — Add to Cart, Checkout
- Sales History — View Orders, Update Status
- Low Stock Alerts
- Pharmacist Management — Add Pharmacist, View Stats

Legend

PASS	Expected behaviour achieved
FAIL	Expected behaviour not achieved

Test Cases

TC-2.1 User Sign-Up Flow

Field	Details
Test Case ID	TC-01
Module	Authentication — SignUpActivity
Priority	High
Preconditions	App launched; no existing account with test email
Test Objective	Verify a new customer can create an account successfully via Firebase Auth and have their profile stored in Firestore and local Room DB.
Test Data	Name: <i>Ali Raza</i> Email: <i>ali.raza@test.com</i> Phone: <i>03001234567</i> Password: <i>Test@1234</i> Confirm: <i>Test@1234</i>
Steps	1. Launch app → tap Get Started on Landing screen 2. On Sign Up screen, enter all fields 3. Tap SIGN UP
Expected Result	• Firebase user created with <code>UserRole.USER</code> • User document written to Firestore <code>users</code> collection • User inserted into Room DB (<code>UserDao</code>) • <code>UserPrefs.isLoggedIn = true</code> • Navigate to <code>MainActivity</code>; toast "Welcome Ali Raza!"
Negative Cases	• Empty fields → toast "All fields are required" • Mismatched passwords → toast "Passwords do not match" • Duplicate email → Firebase error displayed in <code>tvError</code>

TC-2.2 User Sign-In Flow

Field	Details
Test Case ID	TC-02
Module	Authentication — SignInActivity
Priority	High
Preconditions	Account exists in Firebase for test email
Test Objective	Verify registered users (Pharmacist / Customer) can sign in; their role is fetched from Firestore and saved locally.
Test Data	Email: <i>pharmacist@test.com</i> Password: <i>Test@1234</i>
Steps	1. Open Sign In screen 2. Enter email and password 3. Tap SIGN IN
Expected Result	• Firebase <code>signInWithEmailAndPassword</code> succeeds • <code>User</code> object fetched from Firestore and cached in Room • <code>UserPrefs.setLoggedIn(true)</code> called • Redirected to <code>MainActivity</code> with role-appropriate UI
Negative Cases	• Wrong password → Firebase error in <code>tvError</code> • Empty fields → toast "Please fill all fields"

TC-2.3 Admin Hardcoded Login

Field	Details
Test Case ID	TC-03
Module	Authentication — <code>SignInActivity.loginAsAdmin()</code>
Priority	High
Preconditions	None
Test Objective	Verify the hardcoded Admin account bypasses Firebase Auth and logs in with <code>UserRole.ADMIN</code> , showing the admin-specific dashboard.
Test Data	Email: <i>admin@gmail.com</i> Password: <i>Admin123</i>
Steps	1. Open Sign In screen 2. Enter admin credentials 3. Tap SIGN IN
Expected Result	• <code>loginAsAdmin()</code> invoked without Firebase call • <code>User(uid="ADMIN_ID", role=ADMIN)</code> inserted into Room • <code>MainActivity</code> shows: <i>Add Pharmacist, Pharmacist Performance</i> cards • Billing card hidden; FAB hidden

TC-2.4 Add New Medicine (Pharmacist)

Field	Details
Test Case ID	TC-04
Module	Inventory — AddEditMedicineActivity
Priority	High
Preconditions	Logged in as Pharmacist
Test Objective	Verify a pharmacist can add a new medicine; it is persisted in Room and synced to Firestore <code>medicines</code> collection.
Test Data	Name: <i>Panadol Extra</i> Category: <i>Analgesic</i> Price: <i>120.00</i> Stock: <i>200</i> Unit: <i>tablets</i> Expiry: <i>2026-12-31</i> Min Stock: <i>20</i>
Steps	1. From Inventory, tap FAB (+) 2. Fill all required fields 3. Select expiry date via DatePicker 4. Tap Save Medicine
Expected Result	• <code>MedicineRepository.insert()</code> called • Room DB record created with auto-generated id • Firestore doc written at <code>medicines/{id}</code> • Toast "Medicine added!" shown; navigates back to Inventory • New medicine appears in <code>rvMedicines</code> list
Negative Cases	• Empty name → <code>tilName</code> shows "Required" error • Empty price → <code>tilPrice</code> shows "Required" error • No expiry date → toast "Select expiry date"

TC-2.5 Edit Existing Medicine

Field	Details
Test Case ID	TC-05
Module	Inventory — AddEditMedicineActivity (edit mode)
Priority	Medium
Preconditions	At least one medicine exists; logged in as Pharmacist or Admin
Test Objective	Verify editing a medicine updates the record in both Room and Firestore while preserving the original <code>addedBy</code> field.
Steps	1. Tap Edit on a medicine card in Inventory 2. Modify the price and stock fields 3. Tap Save Medicine
Expected Result	• Activity launched with <code>medicine_id</code> extra; fields pre-populated • <code>MedicineRepository.update()</code> called • <code>lastUpdatedBy</code> set to current user UID • <code>addedBy</code> preserved from original record • Firestore doc updated at same <code>medicines/{id}</code> path • Toast "Medicine updated!" shown

TC-2.6 Billing — Add to Cart and Checkout

Field	Details
Test Case ID	TC-06
Module	Billing — <code>BillingActivity</code>
Priority	High
Preconditions	At least 2 medicines with stock > 0; logged in as Pharmacist
Test Objective	Verify medicines can be added to cart, quantities adjusted, and a checkout creates a Sale record with stock deducted.
Test Data	Medicine A: <i>Panadol Extra</i> , Qty: 2 Medicine B: <i>ORS Sachet</i> , Qty: 1 Customer Name: <i>Usman Khan</i>
Steps	1. Open Create Order from dashboard 2. Search and tap + cart button for Medicine A 3. Add Medicine B similarly 4. Adjust quantities using +/− buttons in cart 5. Enter customer name 6. Tap Checkout
Expected Result	• Cart total updates with each quantity change • <code>SaleRepository.insertSaleWithItems()</code> called • Sale record saved in Room with status COMPLETED • <code>SaleItem</code> records created for each cart item • Stock decremented in Room and Firestore for each medicine • Navigate to <code>ReceiptActivity</code> showing correct totals
Negative Cases	• Medicine with stock=0 → toast "out of stock" • Quantity exceeds stock → toast "Not enough stock" • Empty cart → Checkout button disabled

TC-2.7 Sales History — View and Update Order Status

Field	Details
Test Case ID	TC-07
Module	Sales History — SalesHistoryActivity
Priority	High
Preconditions	At least one Sale exists; logged in as Pharmacist
Test Objective	Verify sales are displayed correctly per role; Pharmacist can update order status which syncs to Firestore.
Steps	1. Open Sales History from dashboard 2. Verify orders listed with ID, date, total, status 3. Tap Update Status on an order 4. Select COMPLETED from dialog
Expected Result	• All pharmacist orders displayed in <code>rvSales</code> • Revenue card shows sum of COMPLETED/PAYMENT_DONE orders • <code>SaleRepository.updateStatus()</code> called • Room DB and Firestore both updated with new status • Toast "Status updated to COMPLETED" shown
Customer Role Variant	• <code>UserRole.USER</code>: only own orders shown • "Cancel Order" button visible for PAYMENT_PENDING orders • Revenue card hidden

TC-2.8 Low Stock Alerts

Field	Details
Test Case ID	TC-08
Module	Low Stock — LowStockActivity
Priority	Medium
Preconditions	At least one medicine where <code>stock ≤ minStock</code> ; logged in
Test Objective	Verify the Low Stock screen correctly identifies and displays medicines below their minimum stock threshold.
Test Data	Medicine: <i>Aspirin 100mg</i> , Stock: 5, MinStock: 20
Steps	1. Open Low Stock Alerts from dashboard 2. Review the alert count banner 3. Review listed medicines 4. Tap Update Stock action chip
Expected Result	• <code>MedicineDao.getLowStockMedicines()</code> returns medicines where <code>stock ≤ minStock</code> • Alert banner shows correct count (e.g. "1 low-stock items") • "LOW STOCK" red badge visible on medicine card • Tapping Update Stock chip navigates to InventoryActivity • If no low stock items: "All medicines are well stocked!" shown

TC-2.9 Add Pharmacist (Admin Flow)

Field	Details
Test Case ID	TC-09
Module	Admin — AddPharmacistActivity
Priority	High
Preconditions	Logged in as Admin
Test Objective	Verify Admin can create a new pharmacist account via Firebase Auth; profile saved to Firestore with <code>UserRole.PHARMACIST</code> .
Test Data	Name: <i>Dr. Sara Ahmed</i> Email: <i>sara.pharmacist@test.com</i> Phone: <i>03219876543</i> Password: <i>Pharma@2024</i>
Steps	1. From Admin dashboard, tap Add New Pharmacist 2. Fill in all fields 3. Tap Create Pharmacist Account
Expected Result	• <code>auth.createUserWithEmailAndPassword()</code> succeeds • User with <code>role=PHARMACIST</code>, <code>addedByAdmin=true</code> written to Firestore <code>users/{uid}</code> • Toast "Pharmacist added successfully!" shown • Activity finishes; returns to Admin dashboard
Negative Cases	• Empty required fields → toast "Please fill required fields" • Duplicate email → Firebase error shown in <code>tvError</code>

TC-2.10 Pharmacist Performance Stats (Admin View)

Field	Details
Test Case ID	TC-10
Module	Admin — <code>PharmacistStatsActivity</code>
Priority	Medium
Preconditions	Logged in as Admin; at least one pharmacist with completed sales
Test Objective	Verify Admin can view per-pharmacist performance stats (revenue, orders, efficiency) fetched from Firestore and filtered by time period.
Steps	1. From Admin dashboard, tap Pharmacist Performance 2. Observe default "Today" filter results 3. Change filter to Last Month using spinner
Expected Result	• Pharmacists fetched from Firestore <code>users</code> where <code>role=PHARMACIST</code> • For each pharmacist: <code>MediQuickApi.getPharmacistStats()</code> queried from Firestore <code>sales</code> • Stats displayed: total orders, completed orders, revenue, efficiency % • List sorted by revenue (descending) • Summary row shows aggregate revenue and total orders • Changing filter triggers re-fetch with updated <code>startDate</code> • Offline fallback: if Firestore fails, Room DB queried instead
Negative Cases	• No pharmacists added → "No pharmacists found" empty state shown

Test Summary Matrix

TC#	Test Case	Module	Role	Priority
01	User Sign-Up Flow	SignUpActivity	Customer	High
02	User Sign-In Flow	SignInActivity	All	High
03	Admin Hardcoded Login	SignInActivity	Admin	High
04	Add New Medicine	AddEditMedicineActivity	Pharmacist	High
05	Edit Existing Medicine	AddEditMedicineActivity	Pharmacist	Medium
06	Billing & Checkout	BillingActivity	Pharmacist	High
07	Sales History & Status	SalesHistoryActivity	Pharm/User	High
08	Low Stock Alerts	LowStockActivity	All	Medium
09	Add Pharmacist	AddPharmacistActivity	Admin	High
10	Pharmacist Stats	PharmacistStatsActivity	Admin	Medium

Priority Distribution:

- **High:** TC-01, TC-02, TC-03, TC-04, TC-06, TC-07, TC-09 (7 cases)
- **Medium:** TC-05, TC-08, TC-10 (3 cases)

Coverage Areas:

- Firebase Authentication (Sign Up, Sign In, Admin bypass)
- Firestore read/write sync (medicines, sales, users)
- Room DB offline persistence
- Role-based UI rendering (Admin / Pharmacist / Customer)
- Input validation and error handling
- Stock management and low-stock detection